Linear Regression

Linear Regression is a **supervised machine learning algorithm** used to predict a **continuous numerical value** by finding the relationship between one or more independent variables (features) and a dependent variable (target).

For example(Predicting):
house price based on area.
salary based on years of experience.
sales based on advertising expenditure

Summary
Linear Regression is a supervised machine learning algorithm used to predict a **continuous numerical value** by fitting the **best-fit straight line** that models the relationship between the independent variable(s) and the dependent variable.

Slope(m)	1.30
Intercept	0

$$= \sum_{n=1}^{n} \frac{(y^i - \hat{y})^2}{n}$$

x	y
1.0	3.0
3.0	5.0
4.0	3.0
6.0	7.5
7.0	5.0
8.0	8.0

m x + c = y

(1.3 * 1) + 0 = 1.3

(1.3 * 3) + 0 = 3.9

(1.3 * 4) + 0 = 5.2

(1.3 * 6) + 0 = 7.8

(1.3 * 7) + 0 = 9.1

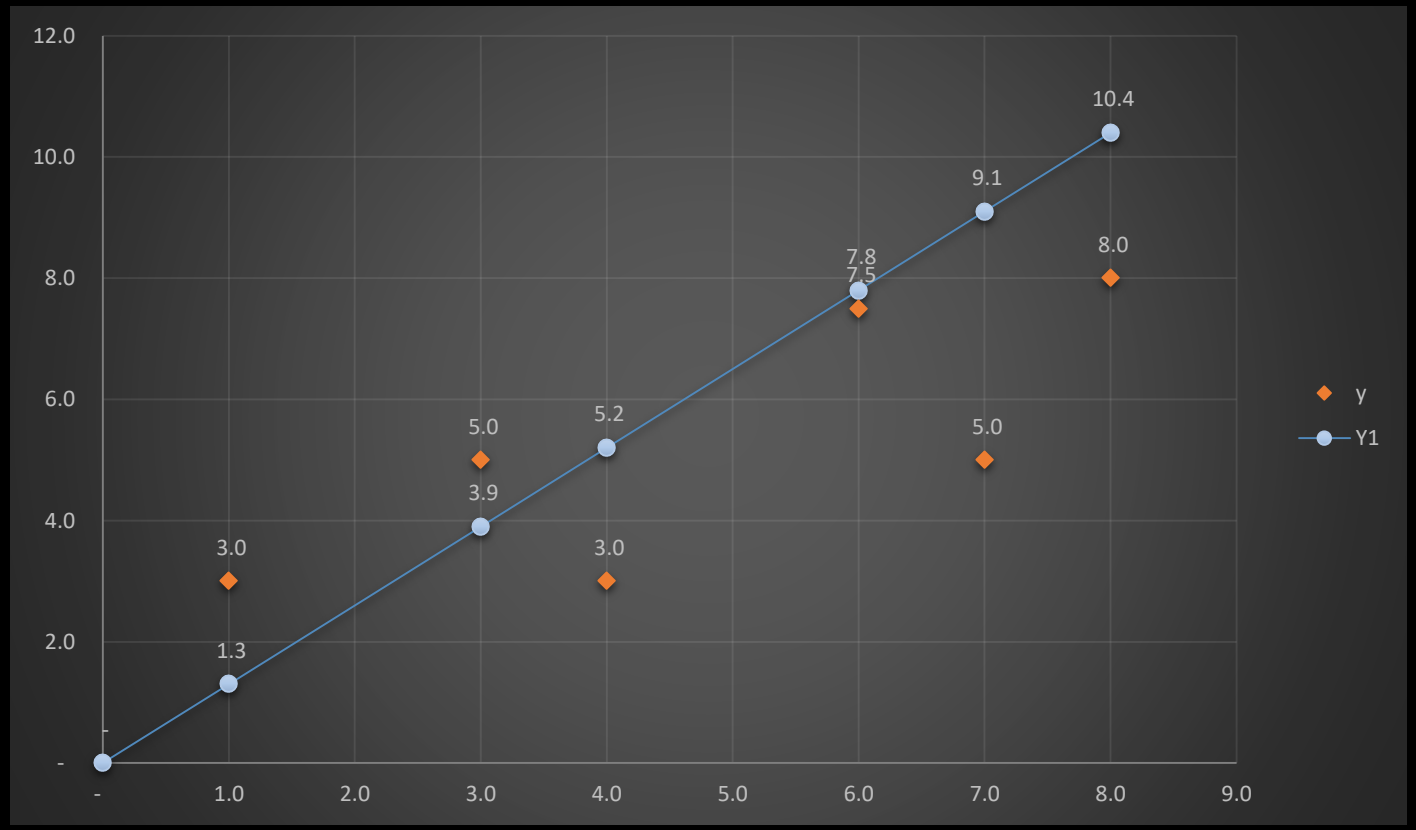
(1.3 * 8) + 0 = 10.4

$$= \frac{(3-1.3)^2+(5-3.9)^2+(3-5.2)^2+(7.5-7.8)^2+(5-9.1)^2+(8-10.4)^2}{6}$$

$$= \frac{(2.7)^2+(1.1)^2+(-2.2)^2+(-0.30)^2+(-4.1)^2+(-2.4)^2}{6}$$

$$= \frac{2.89 + 1.21 + 4.84 + 0.09 + 16.81 + +5.76}{6}$$

$$=5.27$$



Slope(m)	1.02
Intercept	0

$$= \sum_{n=1}^{n} \frac{(y^i - \hat{y})^2}{n}$$

x	y
1.0	3.0
3.0	5.0
4.0	3.0
6.0	7.5
7.0	5.0
8.0	8.0

m x + c = y

(1.02 * 1) + 0 = 1

(1.02 * 3) + 0 = 3.1

(1.02 * 4) + 0 = 4.1

(1.02 * 6) + 0 = 6.1

(1.02 * 7) + 0 = 7.1

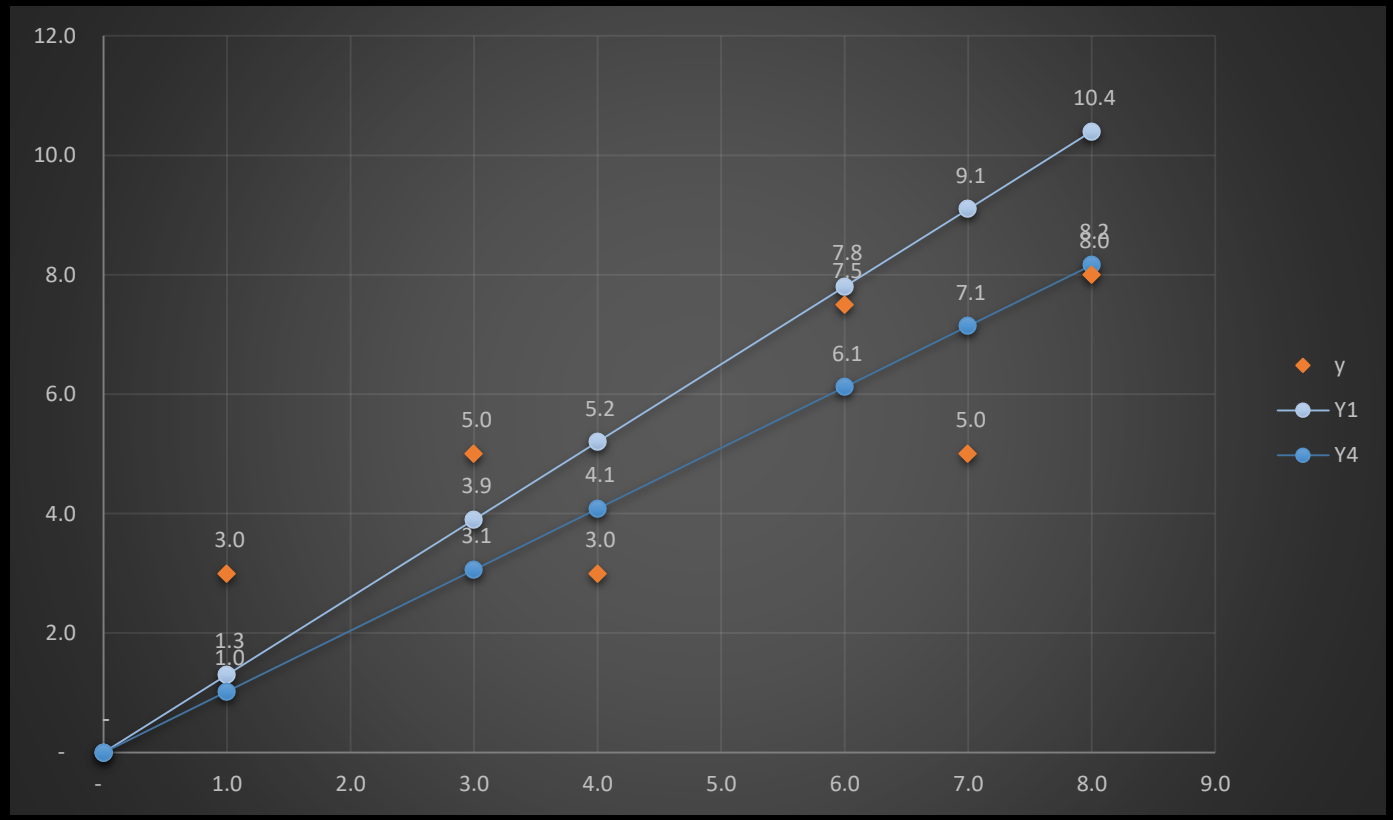
(1.02 * 8) + 0 = 8.2

$$= \frac{(3-1.)^2+(5-3.1)^2+(3-4.1)^2+(7.5-6.1)^2+(5-7.1)^2+(8-8.2)^2}{6}$$

$$= \frac{(2)^2+(1.9)^2+(-1.1)^2+(1.4)^2+(-2.1)^2+(0.2)^2}{6}$$

$$= \frac{4+3.61+1.21+1.96+4.41+0.04}{6}$$

$$=2.53$$



Step 1: Guess a Line:
Step 2: Calculate Errors (Residuals) Compute the Cost

$$residual\ sum\ of\ squares = \sum_{i=1}^{n} (y_i - \hat{y_i})^2$$

Step 4: Change the m and c :
Step 4: Repeat the process :

Step 1: Guess a Line: Start with a guess for a straight line that you think could fit the data. This line is represented by an equation: y = mx + c, where 'm' is the slope and 'c' is the y-intercept.

Step 2: Calculate Errors (Residuals) Compute the Cost For each data point, calculate the vertical distance between the actual y-value (data point) and the predicted y-value. These distances are your errors or residuals.
Calculate the cost function by adding up the squared errors for all data points. The formula might look like the sum of (actual y - predicted y)^2 for each data point. This gives you a single value that indicates how well your guessed line fits the data.

$$residual\ sum\ of\ squares = \sum_{i=1}^{n} (y_i - \hat{y_i})^2$$

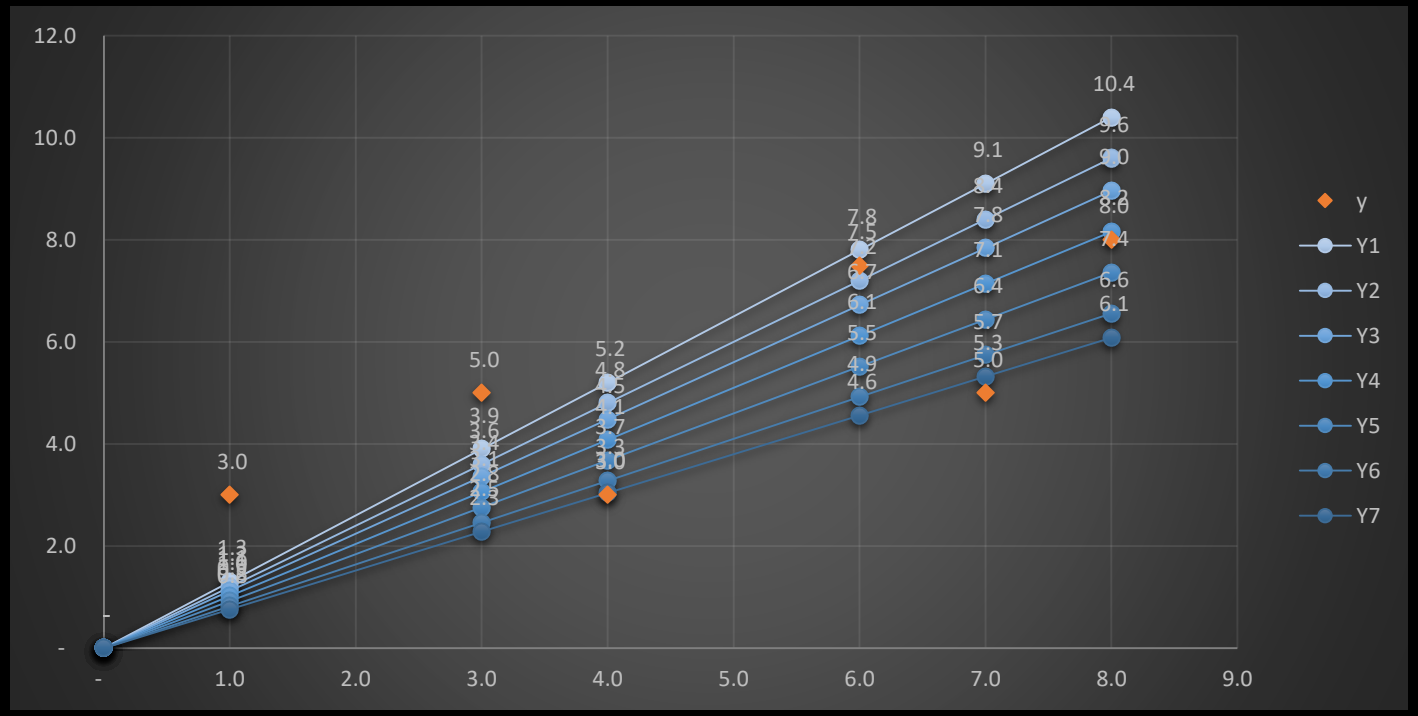
Step 4: Repeat the process : The goal is to minimize the cost function. To do this, you adjust the slope 'm' and the y-intercept 'c' of your guessed line. You want to find the values of 'm' and 'c' that make the cost function as small as possible.

Step 5: Gradient Descent: If using gradient descent, you calculate how much to change 'm' and 'c' based on the slope of the cost function. This helps you move closer to the minimum cost point with each iteration.

Step 1: Guess a Line:

Slope(m)	1.30	1.20	1.12	1.02	0.92	0.82	0.76
Intercept	0	0	0	0	0	0	0
m x + c = y	Y1	Y2	Y3	Y4	Y5	Y6	Y7
(1.3 * 1) + 0 = 1.3	1.3	1.2	1.1	1.0	0.9	0.8	0.8
(1.3 * 3) + 0 = 3.9	3.9	3.6	3.4	3.1	2.8	2.5	2.3
(1.3 * 4) + 0 = 5.2	5.2	4.8	4.5	4.1	3.7	3.3	3.0
(1.3 * 6) + 0 = 7.8	7.8	7.2	6.7	6.1	5.5	4.9	4.6
(1.3 * 7) + 0 = 9.1	9.1	8.4	7.8	7.1	6.4	5.7	5.3
(1.3 * 8) + 0 = 10.4	10.4	9.6	9.0	8.2	7.4	6.6	6.1

x	y
1.0	3.0
3.0	5.0
4.0	3.0
6.0	7.5
7.0	5.0
8.0	8.0



Step 2: Calculate Errors (Residuals)

Slope(m)	1.30
Intercept	0

x	y	Y1
1.0	3.0	1.3
3.0	5.0	3.9
4.0	3.0	5.2
6.0	7.5	7.8
7.0	5.0	9.1
8.0	8.0	10.4

$$cost\ Function = \sum_{n=1}^{n} \frac{(y^i - \hat{y})^2}{n}$$

$$= \frac{(3-1.3)^2+(5-3.9)^2+(3-5.2)^2+(7.5-7.8)^2+(5-9.1)^2+(8-10.4)^2}{6}$$

$$= \frac{(1.7)^2+(1.1)^2+(-2.2)^2+(-0.30)^2+(-4.1)^2+(-2.4)^2}{6}$$

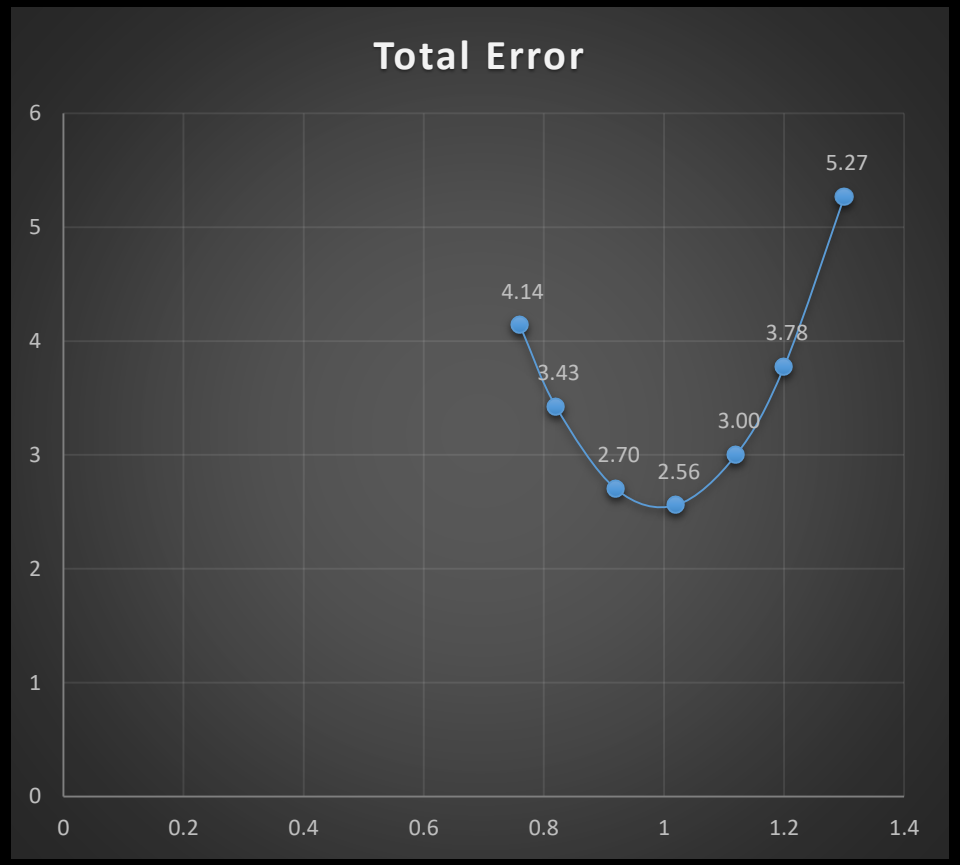
$$= \frac{2.89 + 1.21 + 4.84 + 0.09 + 16.81 + +5.76}{6}$$

$$=5.27$$

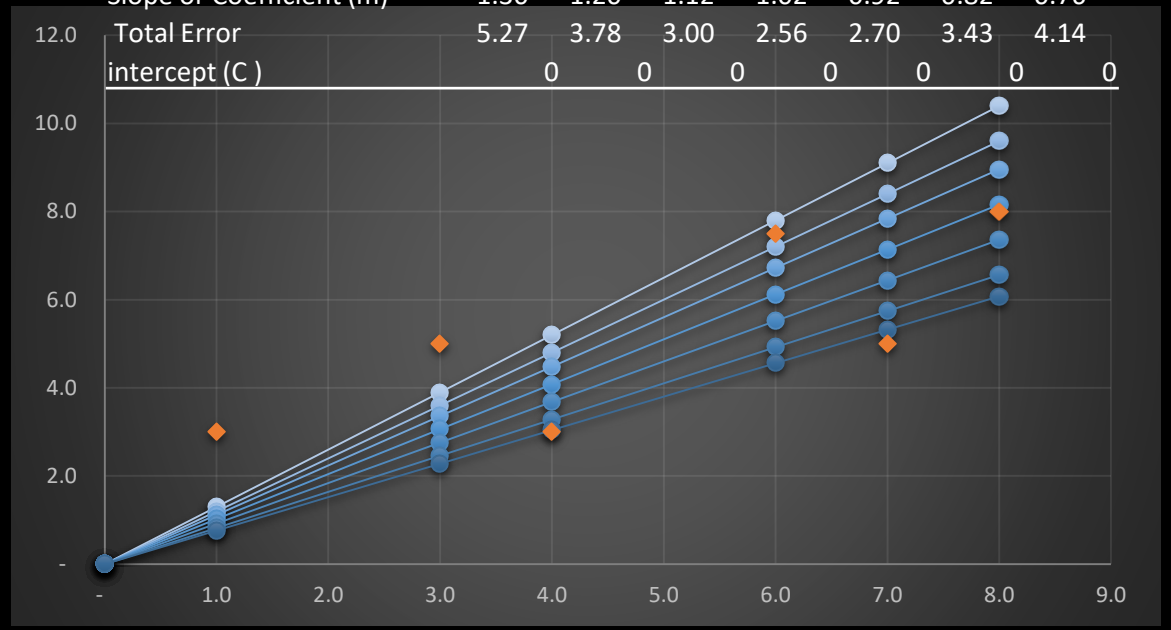
	Y1	Y2	Y3	Y4	Y5	Y6	Y7
Slope(m)	1.30	1.20	1.12	1.02	0.92	0.82	0.76
Total Error	5.27	3.78	3.00	2.56	2.70	3.43	4.14
intercept (C)	0	0	0	0	0	0	0

	Y1	Y2	Y3	Y4	Y5	Y6	Y7
Slope(m)	1.30	1.20	1.12	1.02	0.92	0.82	0.76
Total Error	5.26	3.78	3.00	2.56	2.70	3.43	4.14
intercept (C)	0	0	0	0	0	0	0

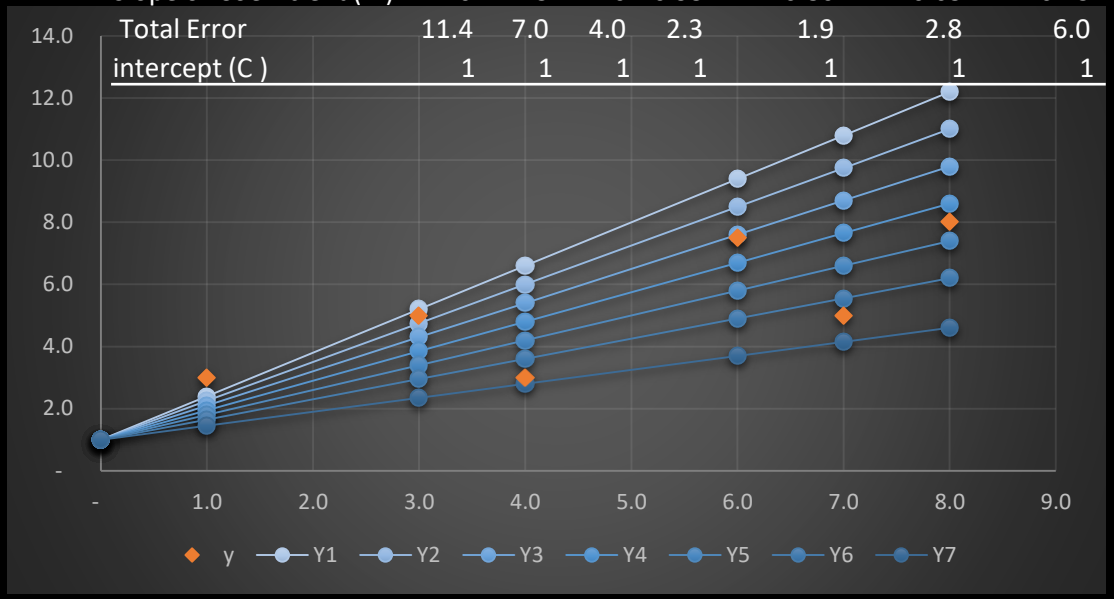
Plot the residuals of 1st Experiment



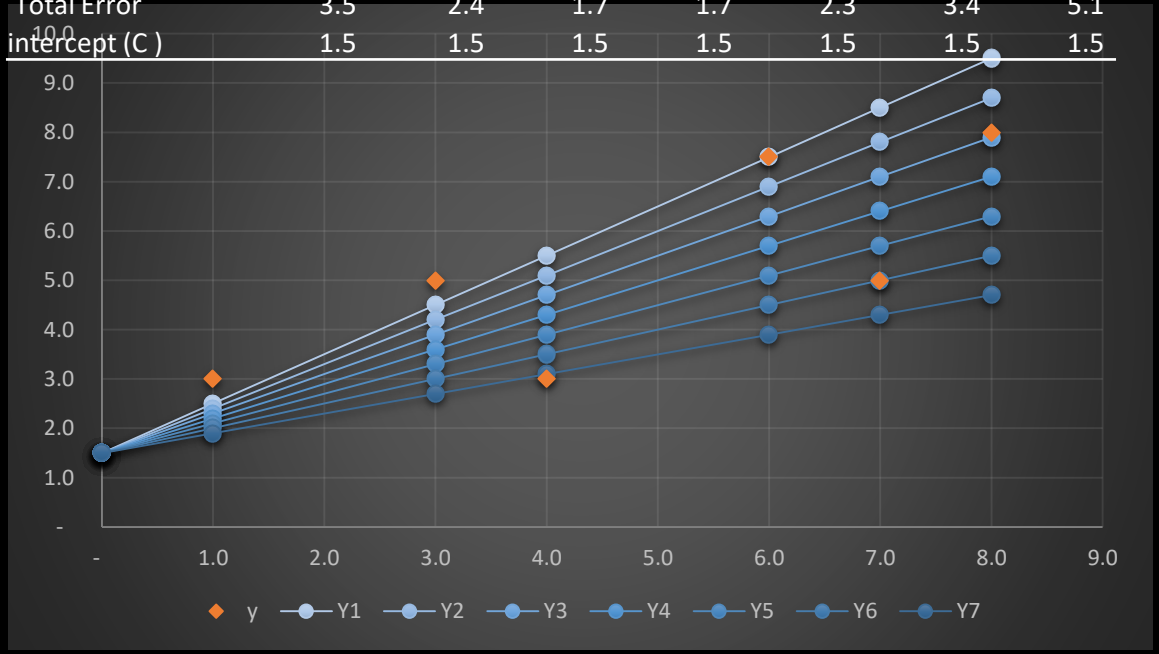
Slope or Coefficient (m)	1.30	1.20	1.12	1.02	0.92	0.82	0.76
Total Error	5.27	3.78	3.00	2.56	2.70	3.43	4.14
intercept (C)	0	0	0	0	0	0	0

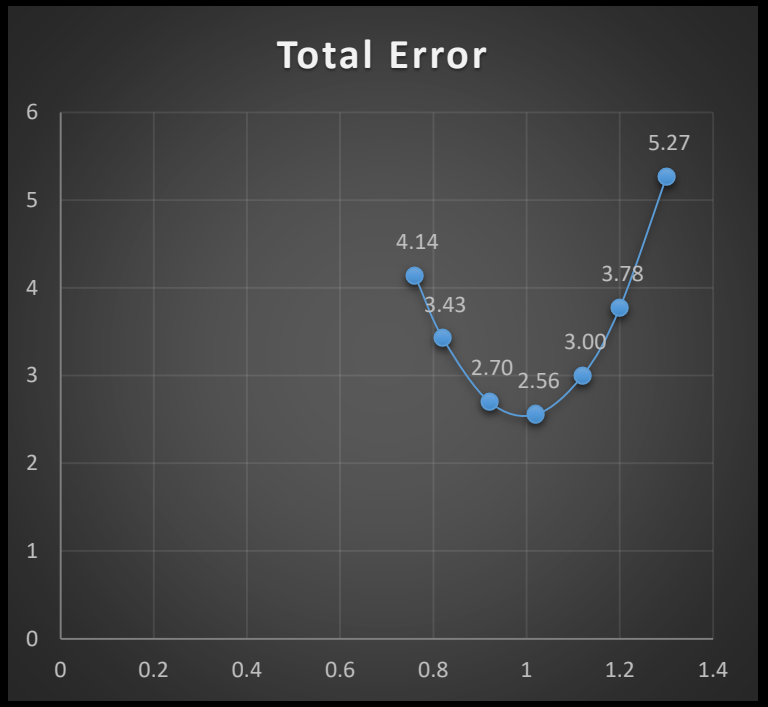


Slope or Coefficient (m)	1.40	1.25	1.10	0.95	0.80	0.65	0.45
Total Error	11.4	7.0	4.0	2.3	1.9	2.8	6.0
intercept (C)	1	1	1	1	1	1	1

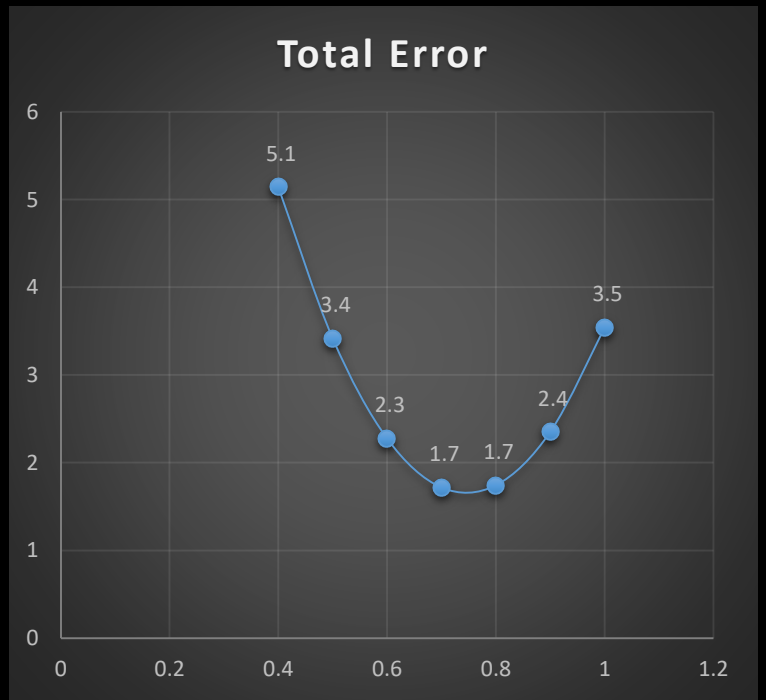


Slope or Coefficient (m)	1.0	0.9	0.8	0.7	0.6	0.5	0.4
Total Error	3.5	2.4	1.7	1.7	2.3	3.4	5.1
intercept (C)	1.5	1.5	1.5	1.5	1.5	1.5	1.5

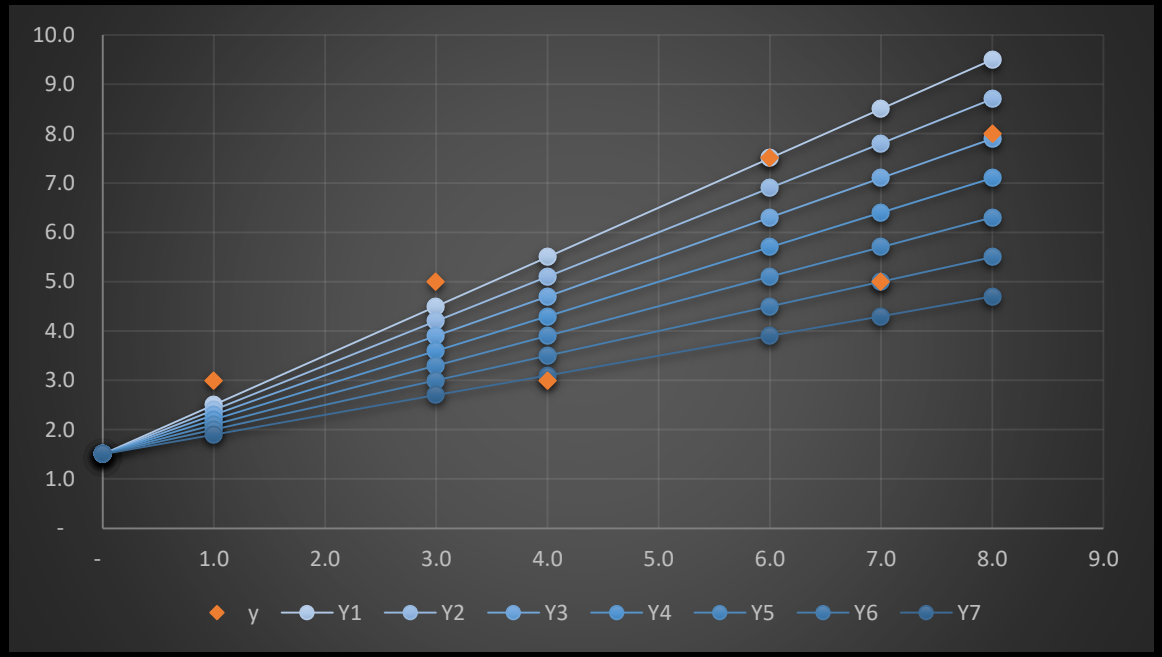



Total Error
6
5
4
3
2
1
0
0
0.2
0.4
0.6
0.8
1
1.2
1.4
4.14
3.43
2.70
2.56
3.00
3.78
5.27

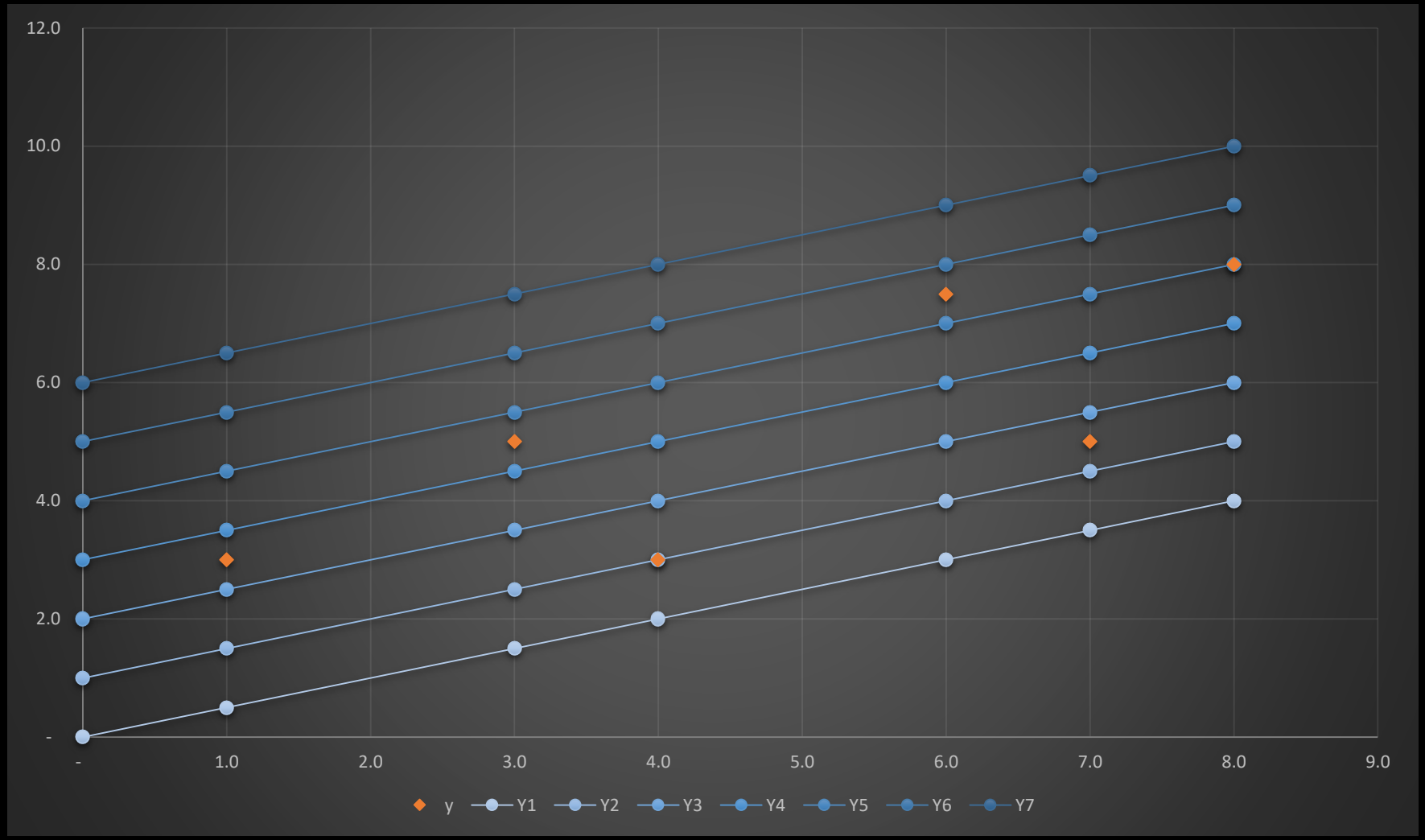

Total Error
12
10
8
6
4
2
0
0
0.5
1
1.5
6.0
2.8
1.9
2.3
4.0
7.0
11.4


Total Error
6
5
4
3
2
1
0
0
0.2
0.4
0.6
0.8
1
1.2
5.1
3.4
2.3
1.7
1.7
2.4
3.5

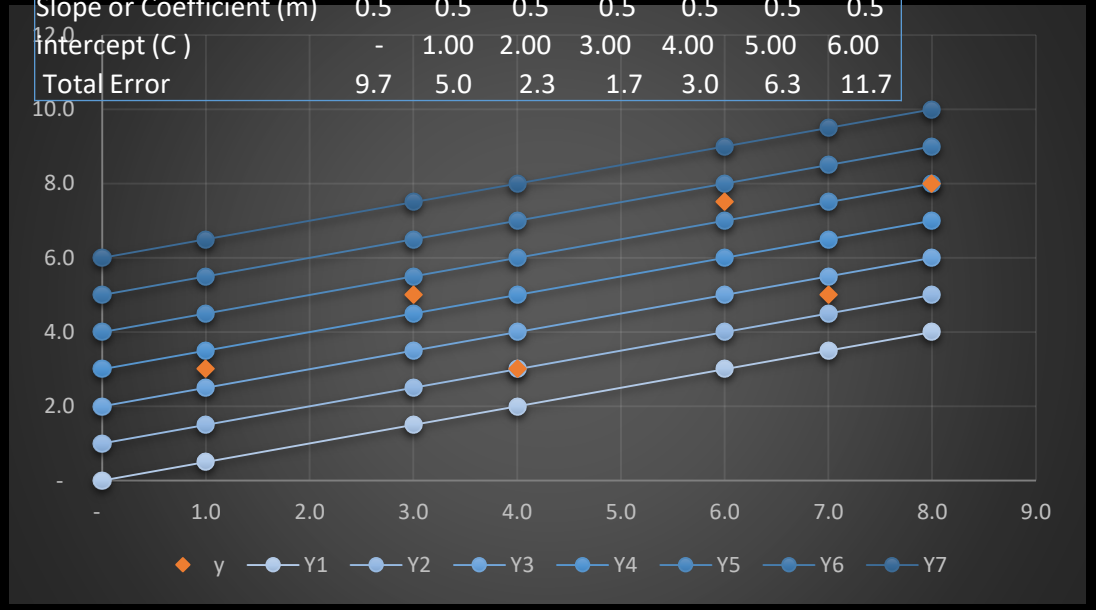
Slope or Coefficient (m)	1.0	0.9	0.8	0.7	0.6	0.5	0.4
Total Error	3.5	2.4	1.7	1.7	2.3	3.4	5.1
intercept (C)	1.5	1.5	1.5	1.5	1.5	1.5	1.5



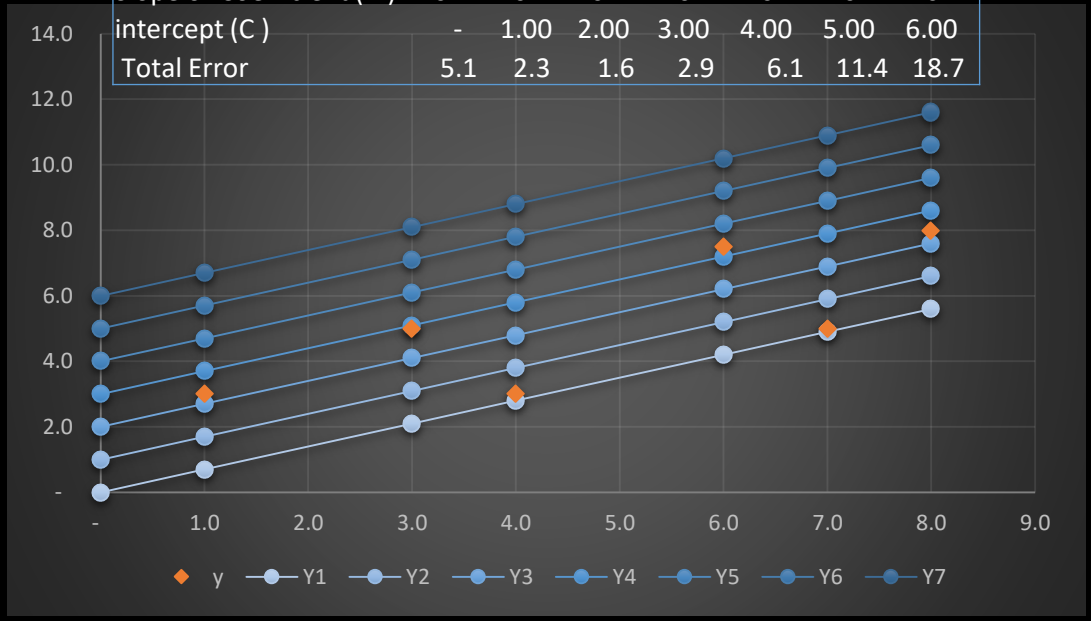
Slope or Coefficient (m)	0.5	0.5	0.5	0.5	0.5	0.5	0.5
intercept (C)	-	1.00	2.00	3.00	4.00	5.00	6.00
Total Error	9.7	5.0	2.3	1.7	3.0	6.3	11.7



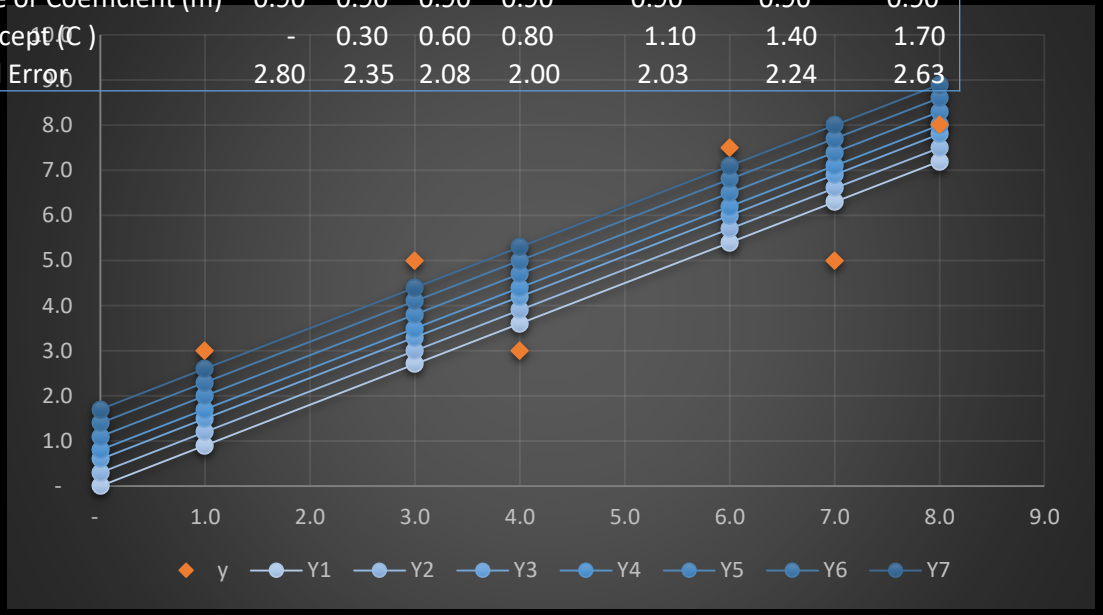
Slope or Coefficient (m)	0.5	0.5	0.5	0.5	0.5	0.5	0.5
intercept (C)	-	1.00	2.00	3.00	4.00	5.00	6.00
Total Error	9.7	5.0	2.3	1.7	3.0	6.3	11.7



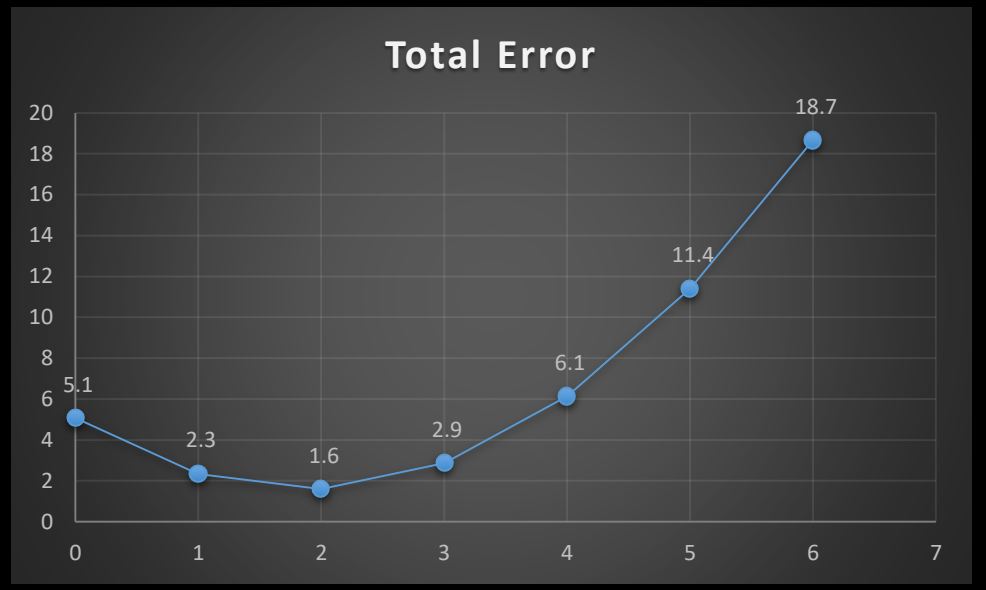
Slope or Coefficient (m)	0.7	0.7	0.7	0.7	0.7	0.7	0.7
intercept (C)	-	1.00	2.00	3.00	4.00	5.00	6.00
Total Error	5.1	2.3	1.6	2.9	6.1	11.4	18.7

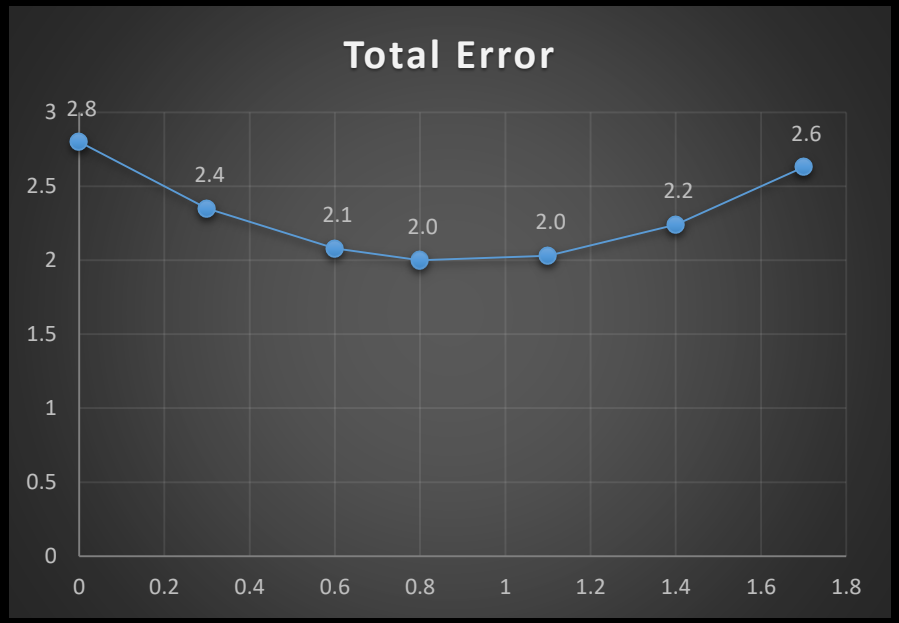


Slope or Coefficient (m)	0.90	0.90	0.90	0.90	0.90	0.90	0.90
intercept (C)	-	0.30	0.60	0.80	1.10	1.40	1.70
Total Error	2.80	2.35	2.08	2.00	2.03	2.24	2.63

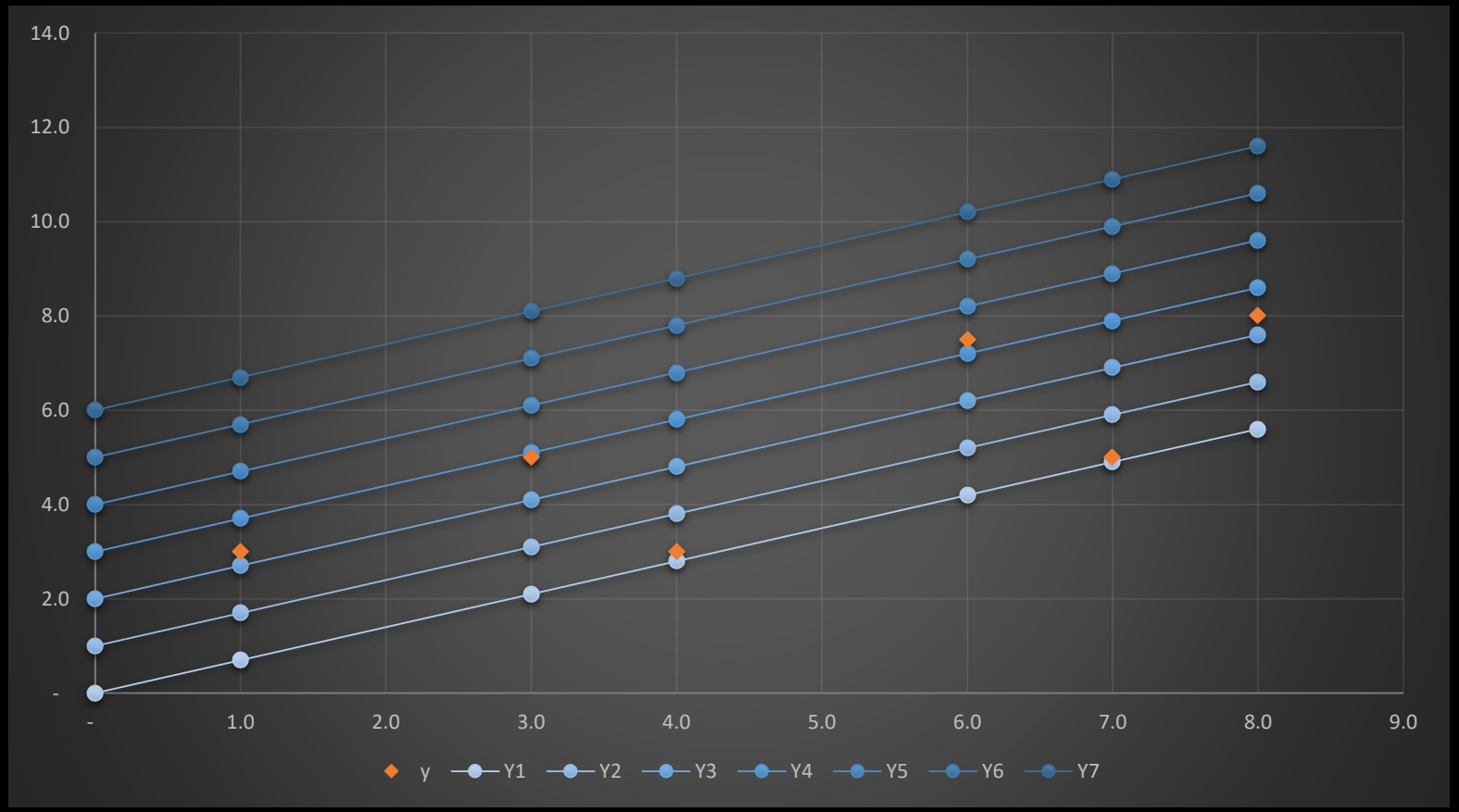



Total Error
14.0
12.0
10.0
8.0
6.0
4.0
2.0
-
9.7
5.0
2.3
1.7
3.0
6.3
11.7
-
1.00
2.00
3.00
4.00
5.00
6.00
7.00

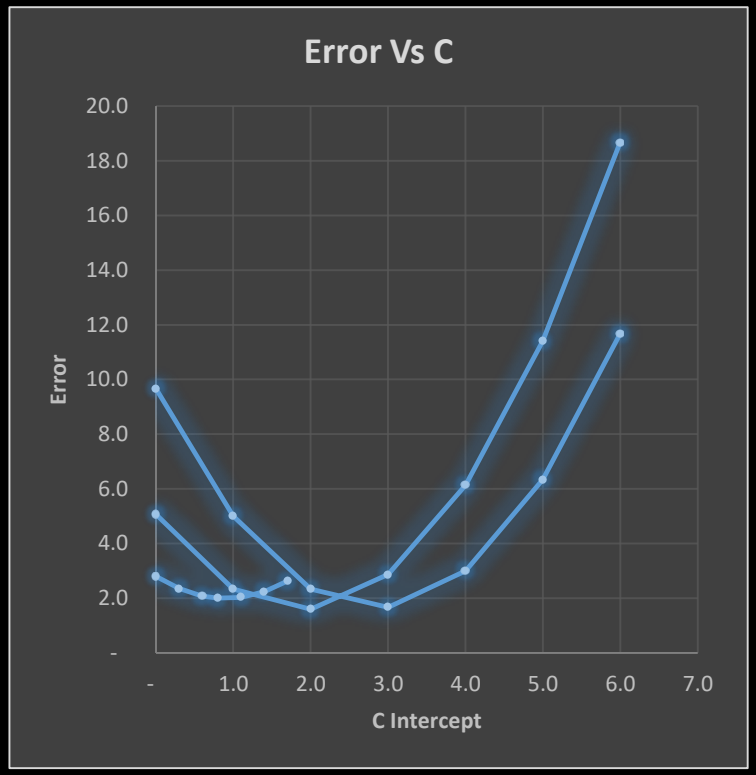

Total Error
20
18
16
14
12
10
8
6
4
2
0
5.1
2.3
1.6
2.9
6.1
11.4
18.7
0
1
2
3
4
5
6
7

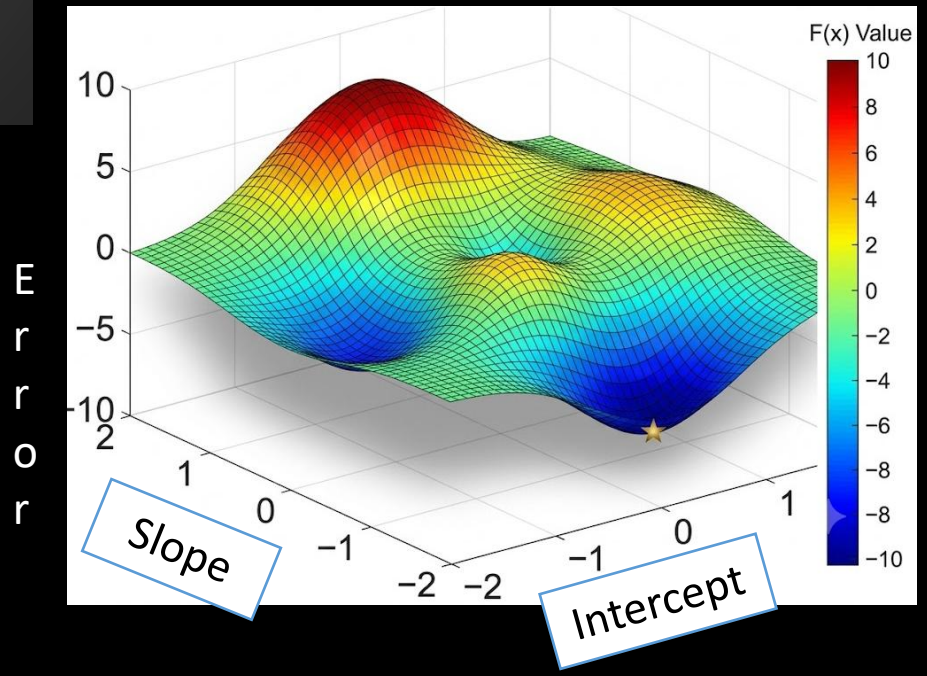

Total Error
3
2.5
2
1.5
1
0.5
0
2.8
2.4
2.1
2.0
2.0
2.2
2.6
0
0.2
0.4
0.6
0.8
1
1.2
1.4
1.6
1.8

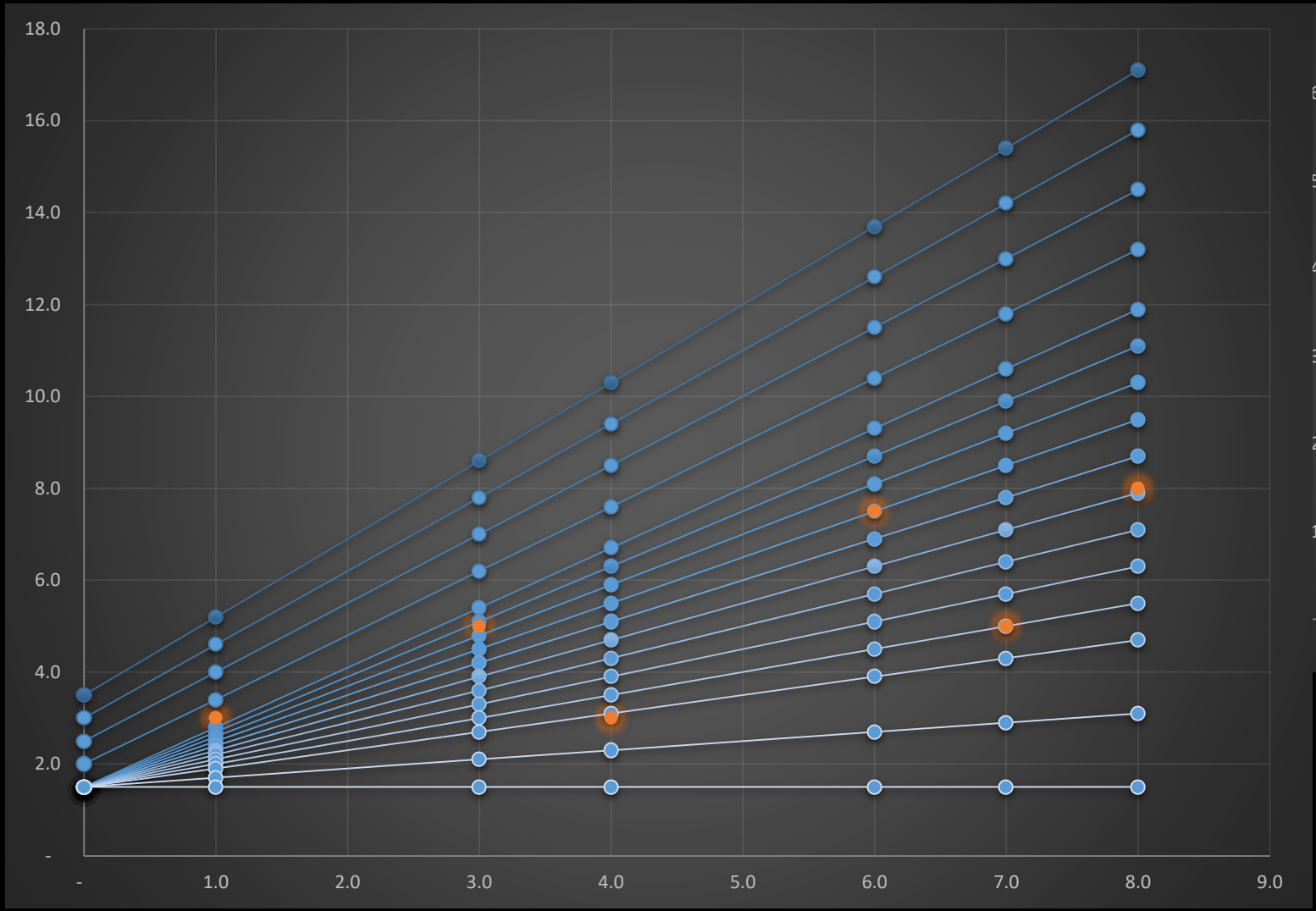
Slope or Coefficient (m)	0.7	0.7	0.7	0.7	0.7	0.7	0.7
intercept (C)	-	1.00	2.00	3.00	4.00	5.00	6.00
Total Error	5.1	2.3	1.6	2.9	6.1	11.4	18.7

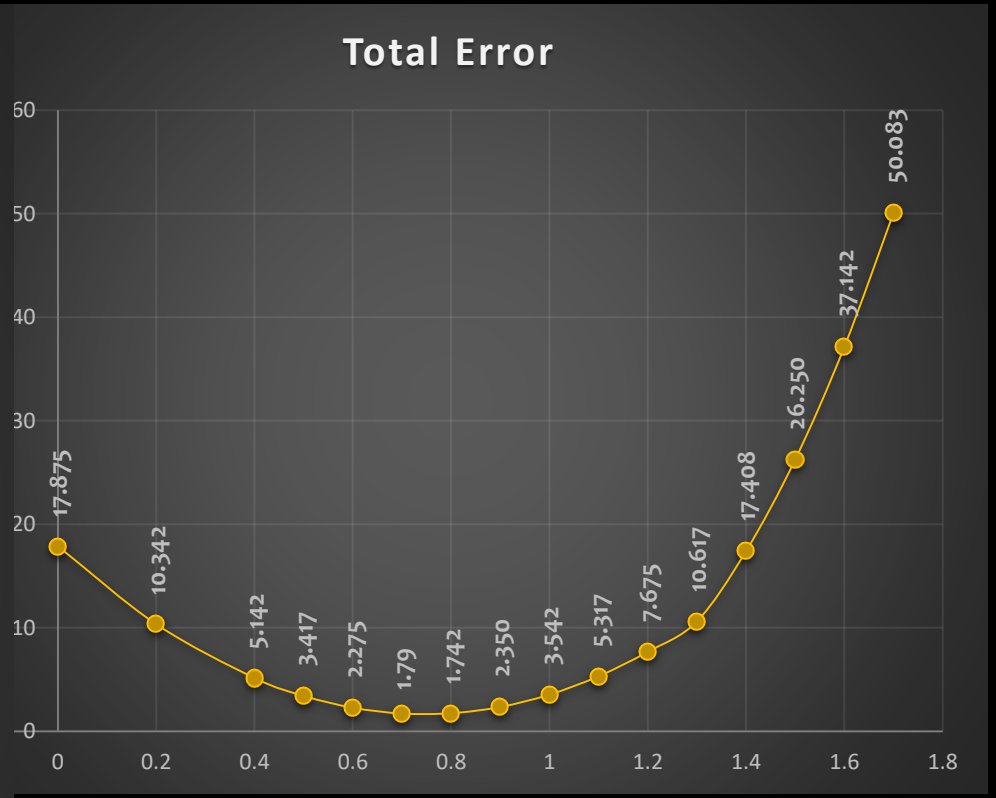



```
from sklearn.linear_model import LinearRegression
```









(C)	(m)	Error
3.5	1.7	**50.1**
3.0	1.6	37.1
2.5	1.5	26.3
2.0	1.4	17.4

(C)	(m)	Error
1.5	1.3	10.6
1.5	1.2	7.7
1.5	1.1	5.3
1.5	1.0	3.5

(C)	(m)	Error
1.5	0.9	2.4
1.5	0.8	1.7
1.5	0.7	1.79
1.5	0.6	2.3
1.5	0.5	3.4
1.5	0.4	5.1
1.5	0.2	10.3
1.5	0.1	17.9

Performance Metrix:

Performance Metrix is used to know the accuracy of the model and they tell you if you're making progress or not. Here we will discuss different types of performance metrics.

1) **MSE (Mean Square Error)**
2) **MAE (Mean Absolute Error)**
3) **RMSE (Root Means Square Error)**
4) **R Square**
5) **Adjusted R Square**

Mean Squared Error:

$$j(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2$$

$\hat{y}_i$ = *Predected Point*
y_i = *Actual Points*
m = *Total Number of points*

Mean Absolute Error

$$j(\theta) = \frac{1}{m}\sum_{i=1}^{m}|y_i - \hat{y}_i|$$

$\hat{y}_i$ = *Predected Point*
y_i = *Actual Points*
m = *Total Number of points*

Root Mean Squared Error:

$$j(\theta) = \sqrt{\frac{1}{2m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2}$$

$\hat{y}_i$ = *Predected Point*
y_i = *Actual Points*
m = *Total Number of points*

Mean Squared Error (MSE):	**Mean Absolute Error (MAE):**	**Root Mean Squared Error (RMSE):**
MSE measures the average of the squared differences between predicted and actual values.	MAE measures the average of the absolute differences between predicted and actual values.	RMSE is the square root of the MSE and shares similar characteristics but is in the same units as the target variable.
It squares the errors, penalizing larger errors more heavily than smaller errors.	It does not square the errors, thus treating all errors equally regardless of size.	RMSE provides a measure of the average magnitude of errors, with the advantage of being in the original units of the target variable.
Because of squaring, MSE is sensitive to outliers.	MAE is less sensitive to outliers compared to MSE. Or robust to outliers	Like MSE, RMSE is also sensitive to outliers.
It's useful when you want to penalize larger errors more significantly or when you're interested in the variance of the errors.	It's useful when you want a metric that's more interpretable and less influenced by outliers.	It's useful when you want to interpret the average error magnitude in the original units of the target variable.
MSE is a convex function. It has a single, global minimum, which means that any local minimum is also the global minimum.	MAE is not convex, It is piecewise linear with multiple linear segments.	RMSE also has a convex function. It has one global minimum.

R Square

R-squared(r2 Square), also known as the coefficient of determination, is a performance metric used to evaluate the goodness of fit of a linear regression model. It provides an indication of how well the model explains the variation in the dependent variable based on the independent variables.

R-squared is a value between 0 and 1. A value of 0 indicates that the model does not explain any of the variability in the dependent variable,

while a value of 1 indicates that the model perfectly explains all the variability.

$$R^2 = 1 - \frac{RSS}{TSS} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

R^2 = coefficient of determination

RSS = sum of squares of residuals

TSS = total sum of squares

R Squared = Small Number / Bigger Number — Then the the output will be in +ve

R Squared = Bigger Number / Small Number — Then the output will be in -ve, and also consider that the worst model is created

Adjusted R Square:

There may be instances where certain features within a model are not correlated with the output feature.

Logically, these features should decrease the accuracy of the model, but in reality, they actually improve the accuracy.

This can lead to an unclear understanding of the model's performance and can result in poor real-time performance if deployed.

To avoid such scenarios, the "adjusted R squared" formula is used.

This ensures that the accuracy of the model decreases when non-correlated features are added and these features are removed during the training phase.

Adjusted R-squared is a modified version of the R-squared that takes into account the number of independent variables in the model.

It is used to assess the goodness of fit of a regression model while penalizing the inclusion of unnecessary independent variables.

$$Adjusted\ R^2 = 1 - \frac{(1 - R^2)(N - 1)}{N - p - 1}$$

Where

R^2 Sample R-Squared

N Total Sample Size

p Number of independent variable

N= Total Sample size (rows)

P= Number of Independent Variable

Independent variables	R-Squared	Adjusted R-Squared
X_1	0.677	0.676
X_1, X_2	0.705	0.703
X_1,X_2,X_3	0.845	0.844
X_1,X_2,X_3,X_4	0.903	0.902
X_1,X_2,X_3,X_4, X_5,	0.972	0.972

Independent Feature						Dependant Feature
x1	x2	x3	x4	x5	x6	y
…	…	…	…	…	…	……
…	…	…	…	…	…	……
…	…	…	…	…	…	……
…	…	…	…	…	…	……
…	…	…	…	…	…	……

Independent Feature						Dependant Feature
x1	x2	x3	x4	x5	x6	y
…	…	…	…	…	…	……
…	…	…	…	…	…	……
…	…	…	…	…	…	……

Overfitting:
Under fitting
Feature selection
Multicolinearity

Overfitting:

If training Data has Very Good Accuracy while Testing Data has Bad Accuracy then this condition is called overfitting.

To solve such a problem need to perform **hyper-parameter tuning, OR Regularization .**

Under-fitting :

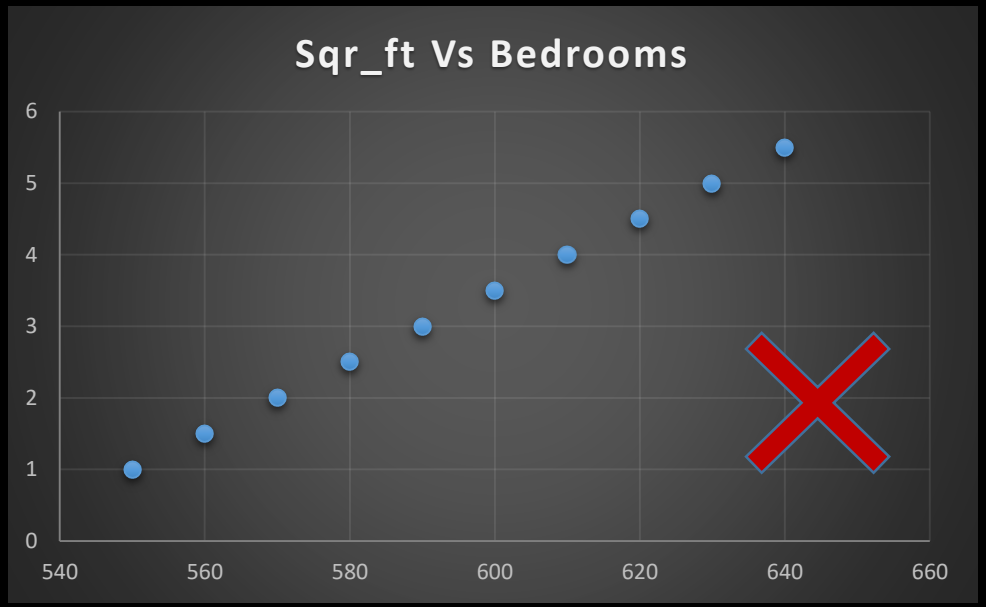
If training Data has Very Bad Accuracy, while Testing Data has Bad or Good Accuracy then this condition is called Under fitting.

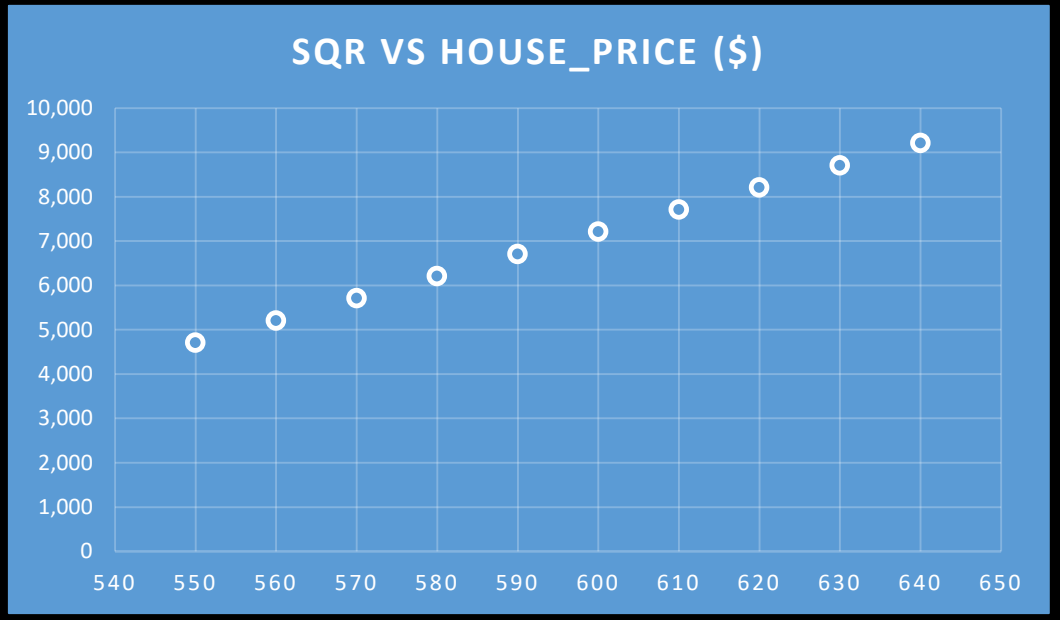
To solve such problem we need to do the retraining of the model.

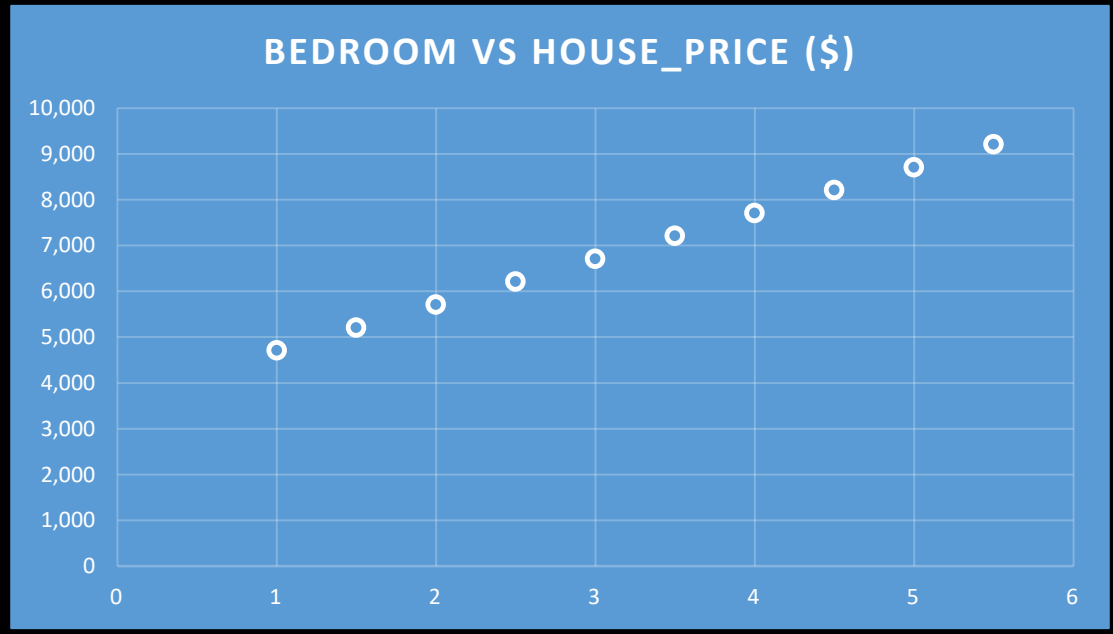
Feature Selection: The process of choosing the most relevant, non-redundant features to simplify models, speed up training.

Multicollinearity:

Square_Feet	Bedrooms	Bathrooms	Age_Years	House Price ($)
550	1	1	21	4,712
560	1.5	1	15	5,212
570	2	2	2	5,712
580	2.5	2	28	6,212
590	3	3	14	6,712
600	3.5	3	26	7,212
610	4	4	7	7,712
620	4.5	4	18	8,212
630	5	5	11	8,712
640	5.5	5	5	9,212







Multicollinearity:
A situation where **two or more independent features** in a dataset are strongly correlated.
This may causing model instability and unreliability in model training.

Multicollinearity is a situation in regression analysis where **two or more independent variables are highly correlated with each other**. This means they contain similar information about the dependent variable, making it difficult for the model to distinguish the individual effect of each predictor.

Square_Feet	Bedrooms	Bathrooms	Age_Years	House Price ($)
550	1	1	21	4,712
560	1.5	1	15	5,212
570	2	2	2	5,712
580	2.5	2	28	6,212
590	3	3	14	6,712
600	3.5	3	26	7,212
610	4	4	7	7,712
620	4.5	4	18	8,212
630	5	5	11	8,712
640	5.5	5	5	9,212

Regularization

- Lasso(L1)
- Ridge(L2)
- Elastic net

Lasso Regression

Think of **Lasso Regression** as a smart version of linear regression that **automatically removes unimportant features**.

Simple example
Imagine you're predicting a house's price using these features:

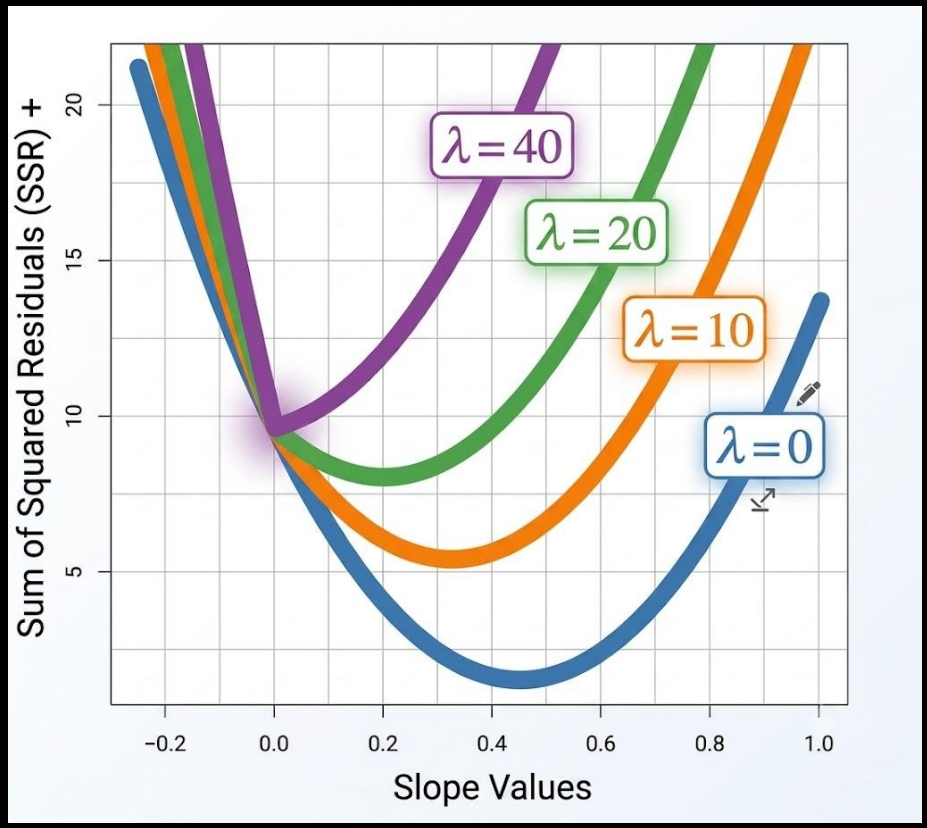
House size	Number of bedrooms	Age of the house	Distance to school	Wall color	Owner's favorite movie
….	….	….	….	….	….
….	….	….	….	….	….
….	….	….	….	….	….
….	….	….	….	….	….

Clearly, **wall color** and **owner's favorite movie** probably don't help predict the price.
A normal linear regression might consider these feature for model building.
However Lasso will ignore such features.

Lasso Regression says:
"If a feature isn't useful, I'll make its slope exactly **0** and ignore it."

Lasso Regression: (L1 Regularization):
Lasso is used to do the **feature selection.**
Lasso adds a penalty term.

$$\lambda \sum_{i=1}^{m} |slope|$$



This penalty term is the sum of the absolute values of the coefficients(slope) which is multiplied by a constant often denoted as **lambda** or alpha. The equation of the same is given below.

Equation after adding the Lasso(L1)

$$(m, c) = \frac{1}{2m} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^{m} |slope|$$

The main aim of the Lasso is to reduce the **number of features** it is used to do the **feature selection.**

Ridge Regression

Think of **Ridge Regression** as a one more smart version of linear regression that **keeps all features in training but reduces their importance**.

Ridge Regression says:
"I'll keep all the features, but I'll make the slope smaller towards 0 does not make exactly to 0
so the model doesn't rely too heavily on any one feature."

Ridge Regression:(L2 Regularization)

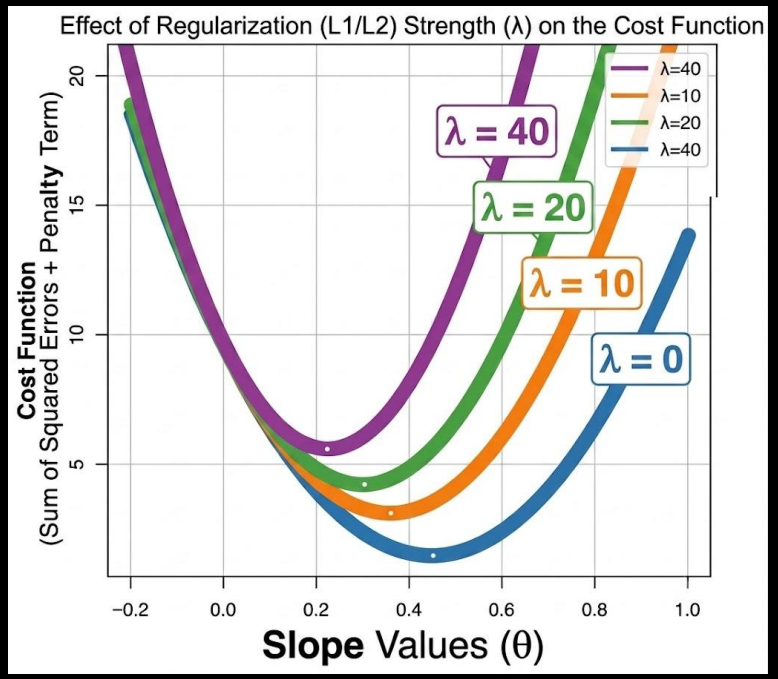
The main purpose of the Ridge is to reduce **overfitting**

When you want to reduce the impact of all features without eliminating them entirely.

$$+ \lambda \sum_{i=1}^{m} (slope)^2$$

Equation after adding the Ridge(L2)

$$(m, c) = \frac{1}{2m} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^{m} (slope)^2$$



Elastic Net Regression:

Elastic Net regression is an extension of linear regression that combines both L1 (lasso) and L2 (ridge) regularization techniques. It aims to overcome some limitations of ridge regression and lasso regression by providing a balance between overfitting multicollinearity and feature selection.

Elastic Net strikes a balance between the two techniques and is useful for high-dimensional datasets with correlated features, offering flexibility in handling multicollinearity while performing feature selection.

$$j(\theta_o, \theta_1) = \frac{1}{2m}\sum_{i=1}^{m}(y_i - \hat{y_i})^2 + \lambda\sum_{i=1}^{m}(slope)^2 + \lambda\sum_{i=1}^{m}|slope|$$

An easy analogy
Imagine five people are carrying a heavy table.

Linear Regression: Some people might carry almost all the weight.

Ridge Regression: Everyone still helps, but the load is shared more evenly.

Lasso Regression: If someone isn't helping much, they are asked to step away completely (their weight becomes 0).

Overfitting:

If training Data has Very Good Accuracy while Testing Data has Bad Accuracy then this condition is called overfitting.

To solve such a problem need to perform **hyper-parameter tuning, OR Regularization .**

Under-fitting :

If training Data has Very Bad Accuracy, while Testing Data has Bad or Good Accuracy then this condition is called Under fitting.

To solve such problem we need to do the retraining of the model.

Overfilling and Under-fitting (Bias and Variance):

Bias :

Accuracy of the training data is mentioned by Bias.

High Bias : If Training Data has Bad Accuracy then.

Low Bias : if Training Data has Very **Good Accuracy**

Variance:

Accuracy of the test data is mentioned by Variance.

High Variance: If Testing Data Bad or Good Accuracy.

Low Variance: If Testing Data Very Good Accuracy.

Overfitting:

If training Data has Very Good Accuracy(Low Bias) while Testing Data has Bad Accuracy (High Variance) then this condition is called overfitting. To solve such a problem need to perform **hyper-parameter tuning.**

Under-fitting :

If training Data has Very Bad Accuracy(High Bias), while Testing Data has Bad (High Variance) or Good Accuracy (Low Variance) then this condition is called Under fitting.